Plan de mantenimiento de la base de datos

Proyecto Final de Desarrollo Web (UMG) — Red social de fotografías. Motor: **PostgreSQL 14+** · Tablas: `users`, `posts`, `followers`, `image_variants`, `face_detections`.

Un mantenimiento regular evita tres problemas: que la base crezca sin control, que las consultas se vuelvan lentas y que un respaldo no sirva cuando se necesita.

1. Calendario de tareas

Frecuencia	Tarea	Responsable
Diaria (automática)	Respaldo con `deploy/backup-db.sh` (cron, 02:00)	Infraestructura
Diaria	Revisar que el respaldo del día aparezca en S3 y que `backup.log` no tenga errores	Infraestructura
Semanal	Revisar espacio en disco, tamaño de tablas y conexiones (sección 3)	Base de datos
Semanal	Revisar `pm2 logs` y `logs/error.log` en busca de errores de BD	Aplicación
Mensual	Buscar y limpiar registros huérfanos (sección 4)	Base de datos
Mensual	Aplicar parches de seguridad: `sudo apt update && sudo apt upgrade`	Infraestructura
Mensual	Revisar índices que no se usan y consultas lentas	Base de datos
Trimestral	**Probar la restauración** de un respaldo en una base de prueba	Base de datos
Antes de cambios de esquema	Respaldo manual + snapshot/AMI	Quien haga el cambio

2. Vacuum y estadísticas

PostgreSQL ejecuta **autovacuum** por sí solo, y para este proyecto basta. Se comprueba así:

```
SELECT relname, n_live_tup, n_dead_tup, last_autovacuum, last_autoanalyze
FROM pg_stat_user_tables ORDER BY n_dead_tup DESC;
```

Si una tabla tiene muchos `n_dead_tup` y `last_autovacuum` está vacío o es muy viejo, forzarlo:

```
VACUUM (ANALYZE, VERBOSE) posts;
```

Reconstruir un índice solo si hay degradación evidente (bloquea escrituras mientras corre):

```
REINDEX TABLE posts;
```

3. Monitoreo semanal

```
df -h /                      # espacio libre en disco (alertar con menos de 20 %)
sudo systemctl status postgresql
```

```
-- Tamaño total de la base
SELECT pg_size_pretty(pg_database_size('mi_red_social'));

-- Tablas más grandes
SELECT relname, pg_size_pretty(pg_total_relation_size(relid)) AS tamano
FROM pg_catalog.pg_statio_user_tables ORDER BY pg_total_relation_size(relid) DESC;

-- Conexiones abiertas
SELECT count(*) FROM pg_stat_activity WHERE datname = 'mi_red_social';

-- Consultas que llevan más de 1 minuto
SELECT pid, now() - query_start AS duracion, left(query, 80) AS consulta
FROM pg_stat_activity
WHERE state = 'active' AND now() - query_start > interval '1 minute';
```

Las imágenes no ocupan espacio en la base (viven en S3); lo que más crece es `image_variants` (4 filas por foto) y `face_detections`.

4. Limpieza de registros huérfanos

`image_variants.owner_id` y `face_detections.owner_id` son referencias **lógicas**: no tienen clave foránea física, porque apuntan a `users` o a `posts` según `owner_type`.

- Al **borrar una publicación desde la app**, el backend elimina sus variantes, sus detecciones faciales y los archivos de almacenamiento (`Controlador/publicacion.js`).
- Si se borra un **usuario** (sus posts se eliminan por `ON DELETE CASCADE`) o se borran filas **directamente en la base**, pueden quedar filas y archivos huérfanos.

Buscarlos:

```
-- Variantes de publicaciones que ya no existen
SELECT v.id, v.storage_key
FROM image_variants v LEFT JOIN posts p ON v.owner_id = p.id
WHERE v.owner_type = 'post' AND p.id IS NULL;

-- Variantes de usuarios que ya no existen
SELECT v.id, v.storage_key
FROM image_variants v LEFT JOIN users u ON v.owner_id = u.id
WHERE v.owner_type = 'user' AND u.id IS NULL;

-- Detecciones faciales huérfanas
SELECT f.id FROM face_detections f LEFT JOIN posts p ON f.owner_id = p.id
WHERE f.owner_type = 'post' AND p.id IS NULL;
```

Procedimiento: (1) guardar la lista de `storage_key`; (2) borrar esos archivos de S3 (o de `imagenes/` si el modo es local); (3) borrar las filas; (4) anotarlo en la bitácora. Hacer un respaldo antes de borrar.

5. Índices

Los índices del esquema ya cubren el feed (`posts(usuario)`, `posts(fecha DESC)`, `image_variants(owner_type, owner_id)`). Para ver cuáles se usan:

```
SELECT relname AS tabla, indexrelname AS indice, idx_scan AS usos
FROM pg_stat_user_indexes ORDER BY idx_scan ASC;
```

Un índice con cero usos durante meses es candidato a eliminarse. Las claves primarias y las restricciones `UNIQUE` **no** se tocan.

6. Cambios de esquema y actualizaciones

- En desarrollo la app usa `sequelize.sync({ alter: true })`; en producción (`NODE_ENV=production`) solo crea tablas faltantes y no altera las existentes.
- Para cambios en producción: respaldo manual, ejecutar un script SQL versionado y anotarlo en la bitácora. Si el proyecto crece, migrar a `sequelize-cli db:migrate`.
- **Actualizaciones menores** de PostgreSQL (14.x a 14.y) llegan con `apt upgrade`; reiniciar fuera de horas de uso y comprobar `pm2 status` después.
- **Actualización mayor** (p. ej. 14 a 16): probarla antes en una instancia nueva con una copia restaurada de un respaldo, nunca directamente en producción.

7. Bitácora de mantenimiento

Cada intervención se anota con esta plantilla:

Fecha	Tarea	Responsable	Resultado	Observaciones
AAAA-MM-DD	Prueba de restauración trimestral			Tiempo que tomó:

8. Lista de verificación mensual

- [ ] Hay un respaldo de cada día del último mes en S3
- [ ] Se probó restaurar uno (trimestral)
- [ ] Espacio en disco por encima del 20 % libre
- [ ] Sin registros huérfanos pendientes de limpiar
- [ ] Parches de seguridad aplicados y `pm2 status` en `online`
- [ ] Bitácora actualizada